

Compiler vlastního programovacího jazyka

Dokumentace k maturitní práci



Ondřej Suk Gymnázium, Praha 4, Písnická 760
12. května 2026

Abstrakt

Práce se zabývá vytvářením jednoduchého compileru programovacího jazyka vytvořeného specificky pro tento projekt za pomoci knihovny LLVM. Jazyk samotný je velmi jednoduchý a je velmi obtížné v něm programovat složitější programy. Důvodem je, že i v aktuálním stavu je projekt poměrně rozsáhlý a tudíž obsahuje jenom minimum, aby v něm bylo možné programovat. Některé části jazyka jsou navrženy netradičním způsobem, což je z důvodu, že na projektu plánuji pracovat i po maturitní zkoušce, tudíž jsem kód přizpůsobil tomu, jak plánuji, aby fungoval finální jazyk.

Compiler jsem implementoval v jazyce C++ 17. Vstupem compileru je soubor s kódem. Ten je potom přeložen do LLVM IR, jazyka poskytovaného knihovnou LLVM, ze kterého je pak zkompilován na objektový kód. Z objektového kódu nakonec vznikne spustitelný soubor. Kompilace z LLVM IR na binární kód je již součástí knihovny LLVM, tudíž se hlavní část projektu zabývá překladem souboru s programem na LLVM IR.

Obsah

1	Úvod	4
2	Programátorská dokumentace	5
2.1	Použité technologie a nástroje	5
2.1.1	Uvnitř projektu	5
2.1.2	Pro vývoj na Linuxu	6
2.1.3	Pro vývoj na Windows	6
2.1.4	Ohledně AI	6
2.2	Základní struktura	6
2.3	Způsob psaní kódu	7
2.4	Kostra	8
2.4.1	Hlavní soubor	8
2.4.2	Třída Module a další pomocné třídy	9
2.4.3	Symbole a Tabulky symbolů	10
2.5	Tokenizace (Lexer)	10
2.6	První fáze parsování (Preparser)	11
2.7	Druhá fáze parsování (Parser)	13
2.8	Generace LLVM IR (Code Generator)	14
2.9	Rozhraní pro LLVM (LLVM Interface)	15
2.10	Problémy s Windows	15
2.10.1	První pokus – Visual Studio	15
2.10.2	Druhý pokus – Terminál	15
2.10.3	Třetí pokus – MinGW	16
2.10.4	Změny v kódu	16
2.10.5	Nedostatky Windows verze	16
3	Uživatelská dokumentace	18
3.1	Instalace a spuštění	18
3.1.1	Kompilace na Linuxu	18
3.1.2	Kompilace na Windows	19
3.2	Uživatelské rozhraní	19
3.3	Funkce	19
3.4	Operátory	21
3.5	Klíčová slova if a while	21
3.6	Proměnné	21
3.7	Ukázkové programy	22
4	Závěr	23

1 Úvod

Jako cíl svého projektu jsem si vymezil tvorbu compileru pro jednoduchý programovací jazyk. Mám v plánu, že na projektu budu pokračovat i po maturitní zkoušce, takže chci, aby šel projekt snadno rozšiřovat a příliš mi nezáleželo na komfortu uživatele jazyka. To znamená, že mým hlavním cílem je implementace základních jazykových konstrukcí, nikoli plnohodnotného jazyka. Mezi věci, které chci implementovat patří smyčky, podmínky, operátory, proměnné, funkce a další.

Následuje programátorská dokumentace. Obsahuje popis obecné struktury projektu a všech systémů, ze kterých se projekt skládá.

Uživatelská dokumentace je méně podrobná než programátorská. Slouží spíše jako ukázka programů, které jdou v jazyce psát, než jako specifikace. Jazyk se stále vyvíjí a informace, které by specifikace obsahovala, by nemuseli platit pro novější verze. Uživatelská dokumentace také popisuje instalační proces a uživatelské rozhraní.

2 Programátorská dokumentace

2.1 Použité technologie a nástroje

Níže je seznam použitých nástrojů. Je členěn na tři části. První oddíl obsahuje nástroje, které mají v projektu viditelnou stopu. Druhý je věnován nástrojům, kterých jsem využil při programování na Linuxu, ale místo nich je k editaci projektu snadné dosadit nástroje jiné. Poslední oddíl se věnuje mému přístupu k umělé inteligenci při práci na projektu.

2.1.1 Uvnitř projektu

LLVM LLVM je knihovna jazyka C++, která slouží jako základ compileru. Definuje jazyk LLVM IR, který je nízkoúrovňovou, ale zároveň platformově nezávislou reprezentací kompilovaného kódu. Compiler tedy převede zdrojový kód jazyka do LLVM IR, které pak knihovna LLVM převede na objektový kód cílové platformy. Jeden z důvodů, proč jsem si projekt vybral je, že jsem se chtěl naučit s LLVM.

LLD LLD je linker, který je součástí projektu LLVM. Linker vytváří z objektového kódu spustitelné soubory. Můj compiler ho zavolá automaticky.

Easyargs Knihovnu Easyargs jsem použil hlavně proto, že poskytuje nenáročné rozhraní pro definování argumentů příkazového řádku. Uživatelské rozhraní nepovažuji za důležitou část projektu, tudíž jsem vybral jednoduchou knihovnu.

Makefile Makefile je systém, pomocí kterého jsem vytvořil kompilační skript pro tento projekt. Vybral jsem si ho, protože na rozdíl od jiných C++ build systémů umožňuje Makefile specifikovat přesně ty příkazy, které budou při kompilaci projektu použity.

MinGW MinGW je port nástrojů příkazového řádku Linuxu pro Windows. Podrobnosti, jak jsem MinGW použil jsou v pozdější části dokumentace.

Clang Jako C++ compiler jsem si vybral Clang, protože je součástí LLVM, a protože poskytuje nástroje Clang Format a Clang Tidy, které jsem také použil.

Clang Format Clang Format je nástroj, který zajišťuje, že všechny kód v projektu je správně zformátovaný.

Clang Tidy Clang Tidy je nástroj, který pomáhá s hledáním chyb v kódu ještě předtím, než je vůbec zkompileován.

Git Pro zálohování změn v projektu jsem využil Git a projekt je veřejně viditelný přes repozitář na GitHubu. [1]

L^AT_EX L^AT_EX je můj primární nástroj pro psaní dokumentace.

2.1.2 Pro vývoj na Linuxu

Neovim Neovim sloužil jako můj primární editor pro psaní kódu, hlavně tedy C++.

Kate Kate sloužil jako můj sekundární editor pro psaní kódu. Psal jsem v něm hlavně kompilační skript a testovací programy. Vybral jsem si ho, protože je výchozím editorem pro KDE desktop.

Kile Kile sloužil jako můj editor pro psaní dokumentace. Vybral jsem si ho, protože je výchozím L^AT_EX editorem pro KDE desktop.

LLDB LLDB je debugger, který jsem použil jako poslední možnost při diagnóze chyb v kódu, převážně segmentačních chyb. Vybral jsem si ho, protože je součástí LLVM.

Arch Linux Arch Linux je Linux distribuce, na které probíhala většina vývoje. Použil jsem ho, protože je již mým primárním operačním systémem.

2.1.3 Pro vývoj na Windows

QEMU Protože nemám doma dostupný počítač s instalací Windows, probíhal téměř veškerý vývoj pro Windows uvnitř virtuálního prostředí Windows 11 v QEMU. Je nutné podotknout, že to nebyla ideální volba a vedla k častému zamrzání obrazovky a dalším nepříjemnostem.

Notepad++ Notepad++ jsem si vybral, protože se jedná o dobře optimalizovaný editor. Nechtěl jsem totiž své již tak omezené Windows prostředí zbytečně zatěžovat.

2.1.4 Ohledně AI

Přestože projekt neobsahuje jediný řádek C++ napsaný AI, měla umělá inteligence v mém projektu zásadní roli. Jako poskytovatele AI jsem si vybral ChatGPT, protože jsem ChatGPT používal už před začátkem projektu. Hlavní roli měla AI v debugování kódu. Umělá inteligence má totiž nadlidskou schopnost hledat hrubé chyby. Dále mi umělá inteligence pomohla pochopit věci, které jsem se v rámci projektu učil nově. AI se hodně podílela na sepsání Makefile.

2.2 Základní struktura

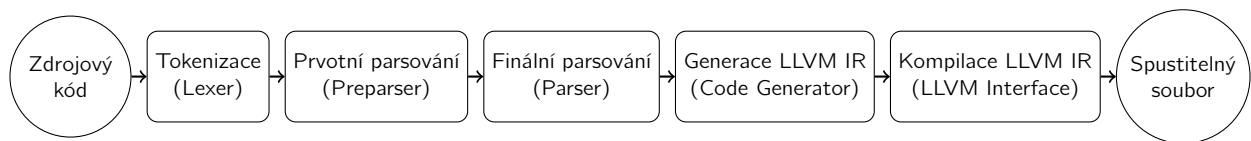
Projekt je rozdělen na dvě hlavní části. První je kostra, která definuje základní běh programu, a druhá jsou jednotlivé fáze, které implementují jednoduché rozhraní pro konverzi kódu z jedné reprezentace na druhou. Tyto reprezentace jsou si poměrně blízké, ale svým vrstvením dokážou zdrojový kód přetvořit na velice rozdílnou formu, spustitelný soubor.

Ve složce s compilerem se nachází několik důležitých souborů. Prvním je `Makefile`, který obsahuje skript, pomocí kterého je možné celý projekt zkompileovat. Dále se zde nachází `.clang-format` a `.clang-tidy`, které obsahují konfigurace pro nástroje používané při vývoji. Méně důležitým souborem je `kernel32.def`, který obsahuje seznam všech Windows API funkcí, které může zkompileovaný projekt zavolat. `kernel32.def` existuje pouze kvůli kompatibilitě s Windows. V poslední řadě tu je soubor `code`, který by vždy měl obsahovat příkladový program k testování projektu.

Všechn zdrojový kód compileru se nachází ve složce `src/`. Soubory, které jsou součástí kostry, a tudíž nepatří žádné kompilační fázi, jsou přímo ve složce `src/` a každá kompilační fáze má vlastní podsložku uvnitř `src/`. Na rozdíl od některých C++ projektů se hlavičkové soubory nachází ve stejné složce jako ostatní soubory s kódem. Soubory s C++ kódem mají přípony `.cpp` a `.hpp`.

Následuje souhrn kompilačních fází se jmény, která mají v kódu, a jejich krátké vysvětlení:

- Lexer – Rozsekává vstupní text na slova. (tokeny)
- Preparser – Parsuje syntaxi klíčových slov a sestavuje tabulku symbolů.
- Parser – Řeší pořadí operátorů a vytváří abstraktní syntaktický strom. (AST)
- Code Generator – Vytváří instrukce LLVM IR na základě AST.
- LLVM Interface – Zkompiluje vzniklé LLVM IR do spustitelného souboru.



Obrázek 2.1: Jednotlivé fáze compileru

2.3 Způsob psaní kódu

Při psaní kódu jsem se snažil být pragmatický. Například jsem nedával příliš velký pozor, zda je všechna alokovaná paměť následně i uvolněna, protože celý compiler běží jen velmi krátkou dobu a všechna paměť je stejně na konci programu uvolněna. Také jsem byl velmi konzervativní s používáním standardní knihovny šablon pro C++, (STL) protože mám zkušenost, že přílišné používání STL může vést k nečitelným chybám a zavádějící logice. Znamenalo to ale, že jsem nevyužil některých užitečných datových struktur, které jazyk C++ nabízí, nebo jsem si je musel implementovat sám. Kód také nekontroluje, zda je uživatelský vstup správný, a jediný kód na nahlášení chyb slouží k vystopování, která část compileru chybu našla, místo aby dal přehlednou zprávu o tom, co se pokazilo. To mi umožnilo na kód rychleji pracovat a dovolilo mi to dokonce některé fáze kompilace jako například Generátor LLVM kódu několikrát celé přepsat, než jsem byl spokojený s výsledkem.

2.4 Kostra

Kód, kterému říkám „Kostra“ řídí běh celého compileru. Při jeho návrhu jsem napřed napsal kód hlavního souboru. Snažil jsem se, aby kód hlavního souboru byl co nejčitelnější. Rozložil jsem to tak, že většina obtížné práce je schovaná za příjemně vypadající metodou `module.build()`. Na druhou stranu jsem věnoval většinu hlavního souboru uživatelskému rozhraní. Místo nesrozumitelného souboru na stovky řádků jsem tedy dostal jednoduchý soubor, pomocí kterého může případný vývojář najít, která část projektu je zodpovědná za jaký úkol. Teprve když byl hlavní soubor hotov, implementoval jsem třídy a metody, které jsem si v hlavním souboru vymyslel. Můj přístup znamenal, že jsem věděl přesně, co musím implementovat. Neztrácel jsem tedy čas psaním zbytečného kódu.

2.4.1 Hlavní soubor

Níže naleznete celý hlavní soubor. Importuje tři knihovny. První, `module.hpp`, je má implementace pomocných tříd, které mi dávají přehledné rozhraní pro interakci s projektem. Pod tím je množství řádků, které definují argumenty příkazového řádku, zakončené importem knihovny `easyargs.h`. Výhodou `easyargs.h` je, že díky chytrému využití preprocesoru jazyka C umí měnit svůj obsah a může tedy vrátit strukturu typu `args_t`, která obsahuje zadanou hodnotu pro každý argument. Poslední knihovnou je `iostream`, což je vestavěná knihovna C++ pro výpis do konzole. Pokud nejste seznámeni s programováním v jazyce C++, nebud'te překvapeni syntaxí `std::cout`, je to způsob, jakým se v C++ píše do konzole. Také je nutné podotknout, že knihovna `Easyargs` je určena pro C, nikoli C++, tudíž jsem musel řešit rozdíly mezi oběma jazyky.

```

#include "module.hpp"
#ifdef __MINGW64__
#define DEFAULT_OUTPUT_PATH "program.exe"
#else
#define DEFAULT_OUTPUT_PATH "program"
#endif
#define REQUIRED_ARGS \
    REQUIRED_STRING_ARG(input_file, "vstup", "Soubor ke kompilaci")
#define OPTIONAL_ARGS \
    OPTIONAL_STRING_ARG(output_file, const_cast<char*>(DEFAULT_OUTPUT_PATH),
        ↪ "-o", "výstup", "Název zkompilevaného souboru")
#define BOOLEAN_ARGS \
    BOOLEAN_ARG(asm_opt, "-a", "Assembly výstup") \
    BOOLEAN_ARG(debug, "-d", "Debug výstup") \
    BOOLEAN_ARG(help, "-h", "Seznam argumentů")
typedef bool _Bool;
#include "easyargs.h"
#include <iostream>
int main(int argc, char* argv[]) {
    // Parse CLI arguments
    args_t args = make_default_args();
    if (!parse_args(argc, argv, &args) || args.help) {
        print_help(argv[0]);
        return 1;
    }
    // Compile to LLVM IR
    Module module{args.input_file};
    IrFile ir = module.build();
    if (args.debug) { // Debug
        std::cout << module << '\n';
        std::cout << ir << '\n';
    }
    if (args.asm_opt) { // Assembly output
        AsmFile instructions = ir.make_asm();
        if (args.debug) // Debug
            std::cout << instructions << '\n';
        instructions.commit(args.output_file);
        return 0;
    }
    // Binary output
    ObjFile obj = ir.make_obj();
    obj.link_executable().commit(args.output_file);
}

```

Listing 1: Celý hlavní soubor

2.4.2 Třída Module a další pomocné třídy

Objekt třídy Module reprezentuje program, který se nachází ve fázích kompilace mezi tokenizací a generací LLVM IR. Konstruktor Module žádá pouze soubor s kódem, na kterém jsou nejdříve provedeny první dvě fáze kompilace, v kódu přezdívané Lexer a Preparser. Module se skládá ze dvou částí, hlavní tabulky symbolů, kterou konstruktor dostane z Preparseru, a balíčku objektů

knihovny LLVM, ve kterém se budou ukládat generované LLVM instrukce. Třída `Module` má jedinou metodu a to je `build`. `build` se podívá do tabulky symbolů a zkompileje vše potřebné do LLVM IR. Jako návratovou hodnotu má `build` objekt třídy `IrFile`. Ta je jednou z několika tříd, které mi usnadňují práci se soubory. Přestože se třída jmenuje `IrFile`, nekoresponduje se souborem na disku. Tyto pomocné třídy totiž reprezentují jen dočasnou hodnotu, dokud není zavolána jejich `commit` metoda, která jim teprve přiřadí trvalé místo na disku.

2.4.3 Symboly a Tabulky symbolů

Základní jednotka kódu se v mém jazyce nazývá `Symbol`. Jednotlivé symboly v mém compileru reprezentují instance třídy `Symbol`. Symbolům jsou přiřazeny tabulky symbolů a ke každému `Symbolu` je možné přiřadit pouze jednu tabulku. Tabulka symbolů může také vlastním symbolům přiřazovat jména. Toho využívám v implementaci proměnných a funkcí.

Existuje několik tabulek zároveň. Jednotlivé tabulky mají mezi sebou hierarchickou strukturu. Hlavní tabulka popisuje globálně viditelné symboly, tedy globální proměnné a funkce, zatímco podřízené tabulky popisují lokální symboly.

Každý symbol v sobě ukládá reference na struktury reprezentující ten stejný symbol v určité fázi kompilace. Díky této struktuře nemusím překládat celý program z jedné reprezentace na jinou naráz, ale můžu zkompilevat jen ty, které potřebuji. Můžu například zkompilevat pouze globální funkci `main` a compiler automaticky určí, které symboly zkompilevat.

2.5 Tokenizace (Lexer)

Tokenizace, neboli lexikální analýza probíhá v mém compileru ve fázi, kterou jsem nazval `Lexer`. Její podstatou je ze vstupního ještě nezpracovaného kódu vytvořit seznam elementárních jednotek syntaxe. Syntax mého jazyka se totiž neskládá výhradně ze znaků, nýbrž často z celých slov. Je proto užitečné být schopný číst kód slovo po slovu, místo písmeno po písmenu. Takovým „slovům“ se říká tokeny.¹ Programovací jazyky často vedle textu do tokenů ukládají další informace. Můj jazyk tohle nedělá, proto (mimo jiné) můj compiler nedokáže uživateli říct, na jakém místě v souboru našel syntaktickou chybu, protože ta informace je první věcí, které se zbavil. Samozřejmě to v budoucnosti lze změnit, jenom jsem tenhle nedostatek upozoroval příliš pozdě a před maturitní zkouškou jsem měl větší problémy. `Lexer` byla první část, kterou jsem na projektu dokončil. Od té doby jsem nemusel na `Lexeru` téměř nic měnit, protože již bohatě stačil pro mé potřeby. Podstatou je, že se `Lexer` snaží seskupovat alfanumerické znaky. Pokud `Lexer` narazí na `whitespace`, tak ho vynechá, jenom v tom místě nesmí seskupovat znaky. Speciální znaky mají každý svá vlastní pravidla tokenizace. Pokud uživatel formátuje svůj kód rozumně, neměl by mít s `Lexerem` v jazyce potíže.

¹Tokeny a slova nejsou ve všech kontextech zaměnitelné. Například při programování LLM se dlouhá slova většinou skládají z několika tokenů.

Zdrojový kód, jak jej vidí uživatel

```
fn num(n) : {  
    if (n >= 10) {  
        num(n / 10);  
    };  
    putchar(48+n%10);  
};
```

Zdrojový kód před tokenizací

f	n		n	u	m	(	n	)		:		{						
	i	f		(	n		>	=		1	0	)		{				
		n	u	m	(	n		/		1	0	)	;					
	}	;																
	p	u	t	c	h	a	r	(	4	8	+	n	%	1	0	)	;	
}	;																	

Zdrojový kód po tokenizaci

fn	num	(	n	)	:	{												
if	(	n	>=	10	)	{												
num	(	n	/	10	)	;												
}	;																	
putchar	(	48	+	n	%	10	)	;										
}	;																	

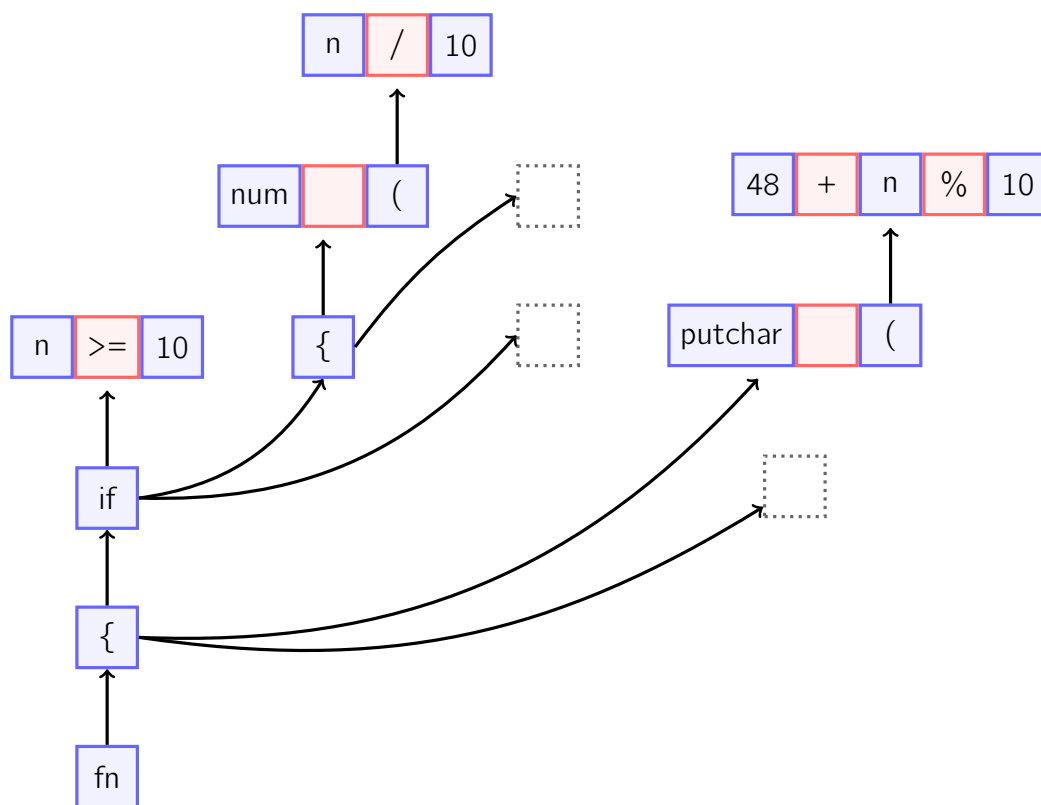
Obrázek 2.2: Rozdíl kódu před a po tokenizaci

2.6 První fáze parsování (Preparser)

Parsování neboli syntaktická analýza rozděljuje program na jednotlivé mezi sebou propojené operace. V této části je nutné poznamenat, že není vůbec běžné rozdělovat parsování na dva kroky a Preparser není obecně zavedený pojem. Parsování jsem rozdělil na dva kroky, protože je pro moje budoucí plány důležité, abych měl co nejdříve představu o názvech symbolů. Proto jsem vytvořil tuto fázi, která sestaví tabulku symbolů hned jako druhý krok. Aby dokázala zjistit věci jako jsou třeba názvy proměnných, je potřeba analyzovat syntaxi klíčových slov. Aby se nemusela opakovat práce, uloží si tato fáze syntaxi libovolného klíčového slova ve

strukturovaném formátu. Aby dále usnadnil dalším fázím práci, přidá Preparser nově vytvořeným symbolům značky podle toho jestli se jedná o operátor a případně o jaký druh operátoru se jedná. Preparser má 4 známé druhy symbolů: hodnota, prefix operátor, infix operátor, postfix operátor. Prefix operátory jsou operátory, které operují jenom na hodnotě za nimi. Příkladem prefix operátoru je negace ($!x$), nebo předpona minus ($-x$). Postfix operátory naopak operují na hodnotě před nimi. V programování jsou celkem vzácné. Příkladem by byl post-decrement operátor ($x--$). Nejčastějším druhem operátoru jsou infix operátory, které operují na obou sousedních hodnotách. Příkladem je odčítání ($x - y$). Pokud se jedná o složitější operátor, jako jsou například závorky, nedokážeme jej pomocí těchto čtyř druhů samotných popsat. Musíme se tedy spolehnout na kód pro řešení klíčových slov, který naopak dokáže popsat téměř jakoukoli syntaxi. Proto jsou závorky klíčovým slovem. Všechny hodnoty musí být odděleny infix operátory. Aby to Preparser zajistil, vloží prázdné infix operátory tam, kde je to potřeba.

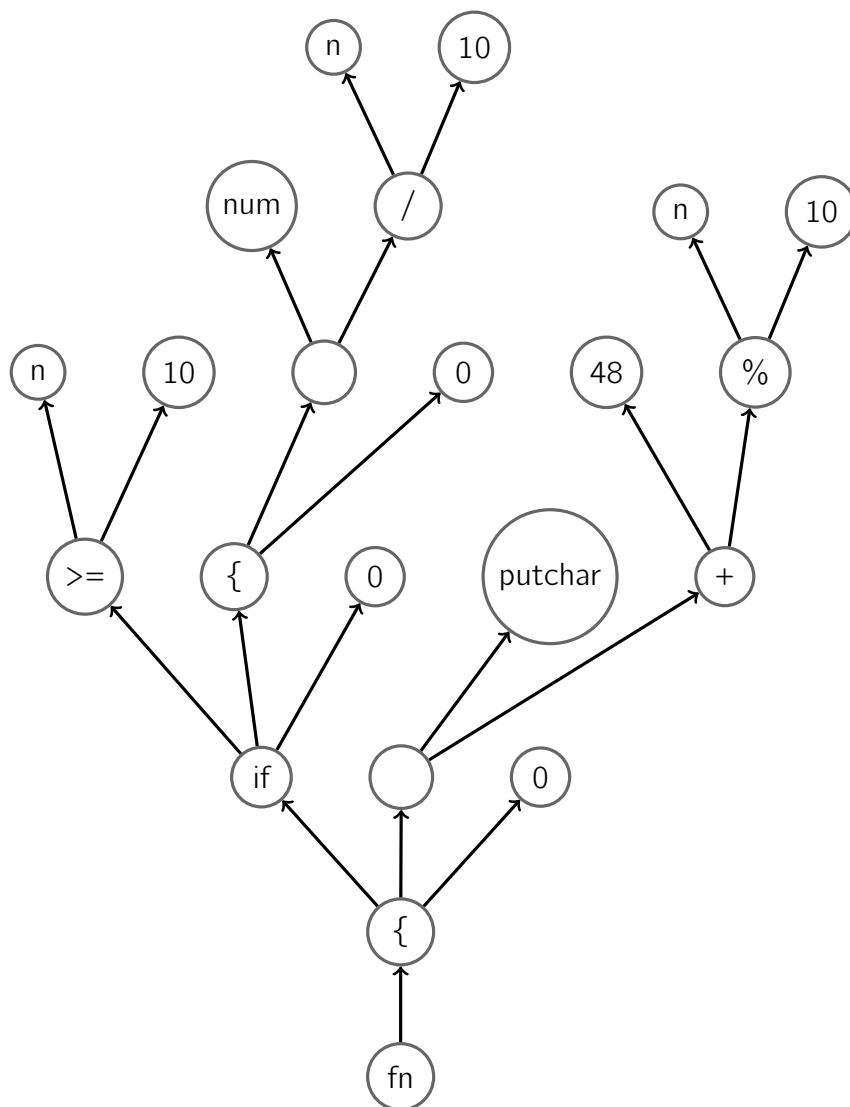
Níže vidíte stejný program jako předtím, který však již prošel Preparserem. Není to optimální příklad, protože obsahuje pouze hodnoty a infix operátory. Také nevím, jestli do maturity přidám závorky jako postfix operátor pro volání funkce, nebo to bude opravdu vypadat jako na obrázku. Nicméně graf dole ukazuje, jak vypadá rozložení paměti v Preparseru. Šipky značí, že Symbol, odkud ta šipka vede, má výraz, na který šipka ukazuje, uložený v sobě a má nad ním absolutní kontrolu. Bílé prázdné čtverečky značí prázdný výraz. (0 symbolů)



Obrázek 2.3: Program z předchozí části očima Preparseru

2.7 Druhá fáze parsování (Parser)

V této fázi konečně sestavíme AST. Jedná se o strom uzlů, kde každý uzel znázorňuje nějakou operaci. Díky pilné práci předchozí fáze je sestavení AST téměř bezbolestné. Bohužel je tu ale jeden problém. Pokud se podíváte na obrázek níže, můžete si všimnout, že nemá stejný počet symbolů, jako ten nahoře. Naštěstí jsem si byl toho problému vědom a vyřešil jsem ho. Symboly mají chytrou funkci, díky které se mohou spojovat dohromady, jako například u závorek, které v AST nejsou, nebo je také možné nové symboly přidávat. To je sice trochu nebezpečnější, protože je teoreticky možné, že dřívější fáze si zažádá symbol vytvořený budoucí fází a ten symbol bude mít jen reprezentaci z té pozdější fáze, ale zatím v mém limitovaném testování tato situace nenastala, takže je logické usoudit, že u správně napsaného klíčového slova to nebude problém.



Obrázek 2.4: AST pro stejný program, jako předtím

2.8 Generace LLVM IR (Code Generator)

```
define i64 @num(i64 %0) {
    %2 = icmp sge i64 %0, 10
    %3 = zext i1 %2 to i64
    %4 = icmp ne i64 %3, 0
    br i1 %4, label %5, label %8

5:                                     ; preds = %1
    %6 = sdiv i64 %0, 10
    %7 = call i64 @num(i64 %6)
    br label %9

8:                                     ; preds = %1
    br label %9

9:                                     ; preds = %8, %5
    %10 = phi i64 [ 0, %5 ], [ 0, %8 ]
    %11 = srem i64 %0, 10
    %12 = add i64 48, %11
    %13 = call i64 @putchar(i64 %12)
    ret i64 0
}
```

Listing 2: Funkce num(n) zkompileovaná do LLVM IR

Na rozdíl od parsování nemám pro Generaci LLVM obrázek, který by vše podrobně vysvětlil. Na rozdíl od předchozích fází, kde jsem si mohl vše volně rozhodovat sám, se zde totiž mísí moje názory o strukturování kódu s názory LLVM. Proto byla tato fáze ze všech nejkritičtější a musel jsem ji třikrát přepsat. Dosud totiž mířily všechny šipky pouze jedním směrem, tudíž symboly vůbec neměly ponětí o tom, na jaké větvi se nachází. Pro generaci instrukcí však symbol nepotřebuje znát pouze, na jakých hodnotách operuje, ale i do jaké funkce má svou instrukci napsat, takže musí být schopný se podívat oběma směry. Proto jsem rozdělil tuto fázi na dva kroky.

V prvním kroku procházíme graf a každému navštívenému symbolu řekneme, odkud jsme k němu došli. V prvním kroku zároveň vytváříme proměnné a funkce. Teprve, když už každý symbol ví, co je v obou směrech, mohou symboly doputovat tím řetězem k funkci, ve které se nacházejí a přidat do ní patřičnou instrukci.

Kromě toho komplikovaného procesu si zkušený programátor v kódu výše něčeho všimne. Program nahoře je totiž velmi špatně napsaný. Hned jako první se v něm porovnávají dvě čísla za vzniku logické hodnoty, která je pak zkonvertovaná na číslo, a pak zase na logickou hodnotu. Dále funkce skáče na blok kódu %8, a pak z něj ihned skočí na blok kódu %9. Pokud se podíváte na instrukci phi, je očividné, že jde zjednodušit pouze na 0, a že její vypočtená hodnota není nikdy použita.

Je patrné, že můj compiler zatím při generaci LLVM IR nedělá vůbec žádnou optimalizaci. Nedělá totiž optimalizaci nikde. To je v souladu s mými cíli, protože by implementovat pořádné optimalizace ubíralo čas od implementace mnohem důležitějších věcí.

2.9 Rozhraní pro LLVM (LLVM Interface)

Tohle je poslední fáze. Zajímavá je v tom, že jako jediná probíhá zcela mimo pomocnou třídu `Module`. Je také, alespoň dle mého názoru, nejméně zajímavá. Jediné, co dělá, je že volá knihovnu LLVM, aby zkompilevala daný LLVM IR kód do Assembly, nebo objektového kódu. Případně pak zavolá linker, (já si vybral LLD) který pak ze souboru s objektovým kódem udělá spustitelný soubor. Kromě toho nemá smysl zacházet tady do větších detailů, protože jsem velkou část zkopíroval z tutoriálu na stránkách LLVM[2], takže pokud vás nejen tato část projektu zaujala, zkuste si ho přečíst.

2.10 Problémy s Windows

Následující část obsahuje primárně mé subjektivní názory a mou osobní zkušenost. Není nijak důležitá k pochopení vnitřní struktury mého programovacího jazyka. Důležité je říct, že moje negativní zkušenost s Windows nebyla způsobená mým primárním zaměřením na Linux, i když skutečnost, že nemám neomezený přístup k počítači s Windows, která mě přinutila používat zamrzávající VM, situaci nepomohla. Následuje převyprávění několikaměsíční snahy zkompilevat můj projekt na školním počítači.

2.10.1 První pokus – Visual Studio

Ze začátku jsem se pokoušel zprovoznit kompilaci přes Visual Studio², kanonický nástroj pro vytváření Windows native aplikací. Prvním problémem bylo, že nemohu prostě vložit svůj `Makefile` do Visual Studia a očekávat, že to bude fungovat. Visual Studio podporuje tři build systémy pro C++: `MSBuild`, `CMake` a `NMake`. Jeden z nich jsem simusel vybrat. `MSbuild` jsem ihned zamítl, protože funguje na základě složitě navigovatelných menu, takže to bylo mezi `CMake` a `NMake`. S `CMake` mám předchozí zkušenost, tedy hlavně negativní předchozí zkušenost, zatímco `NMake` je port `Makefile` pro Windows. Bohužel ale `Makefile` a `NMake` jsou dostatečně odlišné, že bych stejně musel mít soubory na kompilaci dva. Protože jsem neměl náladu ani energii, rozhodl jsem se pro `CMake`. Myslel jsem si, že umělá inteligence, která je trénovaná na mnohem populárnějším `CMake` mi dá, alespoň v prvních pár pokusech, alespoň částečně funkční build skript. Ukázalo se ale, že Visual Studio nemá žádný bezbolestný způsob, jak si stáhnout knihovnu LLVM, takže jsem si ji musel stáhnout manuálně, nebo si ji dokonce zkompilevat sám. Nemyslím si, že projekt rozměru mého má právo žádat o uživateli zkompilevat knihovnu velikosti LLVM, takže jsem si začal stahovat LLVM, když jsem dostal nápad, čímž tento pokus oficiálně skončil.

2.10.2 Druhý pokus – Terminál

Můj nápad byl jednoduchý: „Jestli si stejně musím stahovat celé LLVM, co mi tedy poskytuje Visual Studio?“ Uvědomil jsem si totiž, že mimo jiné distribuce LLVM obsahuje Clang, compiler pro C a C++. Jestliže mám tedy svůj kód a C++ compiler, nic mi nebrání prostě Visual Studio přeskočit a rovnou spouštět compiler pomocí terminálu. Co mi poskytuje build systém, který mě stejně odpuzuje, když kompilační skript můžu napsat v jakémkoli skriptovacím jazyce schopném spouštět příkazy přes příkazový řádek. Takže jsem nechal ChatGPT mi vygenerovat BATCH skript, který prostě spustí všechny potřebné kompilační příkazy. Tohle fungovalo.

²Mluvím zde o programu Visual Studio, nikoli Visual Studio Code. Visual Studio Code nezahrnuje C++ SDK.

Důraz zde dávám na minulý čas. Chtěl jsem přinést ukázat svůj projekt na školním počítači. Když jsme však přišli do učebny, zhrozili jsme se. Školní administrace totiž v ten stejný týden (možná dokonce ten předchozí den) aktualizovala počítače v učebně na Windows 11. Když jsme se přihlásili ukázalo se, že upgrade nevyšel úplně hladce; počítače každých několik minut ukázaly modrou obrazovku. To však nebyl můj největší problém. Ten byl v tom, že upgrade odinstaloval Visual Studio.

Proč mi vadilo, že nemám Visual Studio, když jsem ho neplánoval použít? Protože Visual Studio, jako kononický nástroj pro vytváření Windows native aplikací není pouze editorem. Je to správce celé řady nástrojů, které jsou potřeba při vývoji softwaru. Pokud bylo Visual Studio pryč, byly pryč i vývojářské nástroje jazyka C++. Pro zkompileování C++ nestačí, jak jsem si myslel, C++ kód a C++ compiler, ale je potřeba i C++ standardní knihovny, kterou jsem kvůli absenci Visual Studia neměl a k instalaci Visual Studia je třeba administrátorského přístupu. Radši, než vysvětlovat, že potřebuji Visual Studio nainstalované přestože ho neplánuji používat, jsem se rozhodl najít alternativní metodu. Samozřejmě to nebylo mým jedinným důvodem. Celý tento proces měl mnoho nevýhod.

2.10.3 Třetí pokus – MinGW

Navenek vypadá MinGW jako nástroj, který jsem měl použít rovnou. Poskytuje na Windows prostředí unixového příkazového řádku, což by mi umožnilo kompilovat Linux i Windows verze jedním Makefile. Navíc k instalaci nepotřebuje administrátorská práva. Má však i nevýhody. Hlavní nevýhodou z mého hlediska je, že výsledný soubor je závislý na kódu uvnitř MinGW, (zvláště pokud je to zkompileované s MSYS2 runtime) takže jediný způsob, jak ten projekt spustit mimo prostředí MinGW je najít všechny .dll soubory, na kterých zkompileovaný soubor závisí a přesunout je do stejné složky, což není něco, co by měl člověk dělat.

Přesto to byl pravděpodobně ten správný přístup. Díky tomu jsem nemusel udržovat dvě verze kompilačních skriptů v jiných jazycích a byl jsem schopný dát vše potřebné ke kompilaci projektu na jeden přenosný flash disk. Můžu tedy projekt upravovat a kompilovat na libovolném počítači i bez administrátorských práv.

2.10.4 Změny v kódu

V některých částech kódu, hlavně v Code Generatoru a LLVM interface, se vyskytuje kód, který by na Windows nefungoval. Používám proto C preprocesoru, abych mohl poskytnout oběma platformám funkční implementaci. Jeden problém, který se mi vyskytl pouze na Windows je, že C++ knihovna, která mi poskytovala rozhraní mezi mým compilerem a LLD, byla rozbitá a nefungovala, takže jsem specificky pro Windows udělal alternativní implementaci, která prostě volá LLD přes příkazový řádek.

Větším problémem je, že Linux a Windows používají pro komunikaci s operačním systémem velmi odlišné rozhraní. Kvůli tomu jsem musel udělat statickou knihovnu, která dělá jednu věc: importuje 3 Windows API funkce. Program přeložený do objektového kódu pak s knihovnou zkombinuji pomocí linkeru. Na Linuxu mi stačila `syscall` instrukce v Assembly.

2.10.5 Nedostatky Windows verze

Kromě závislosti na MinGW je momentálně největší problém v zobrazování Unicode znaků na obrazovce. Problém to je v C/C++ na Windows dlouhodobý. Windows očekává, že výstup obsahující Unicode znaky mu bude předán v UTF-16, zatímco Linux přijímá UTF-8 bez

problémů. Easyargs je cíleno na unixové prostředí, takže kód očekává 8-bitové znaky, jako v UTF-8. Když se Easyargs snaží předat Windowsu text v UTF-8, Windows odmítne rozpoznat Unicode znaky a napíše místo toho několik náhodně vypadajících znaků.

3 Uživatelská dokumentace

3.1 Instalace a spuštění

Aktuálně projekt podporuje dvě platformy: Linux a Windows. Windows verze funguje díky vrstvě kompatibility MinGW, takže bez ní nelze spustit. Projekt momentálně není ve zkompilevané formě dostupný, takže je nutné si jej zkompilevat sám.

3.1.1 Kompilace na Linuxu

Následuje vysvětlení postupu kompilace pro Linux. Aktuálně není na GitHubu projekt ve zkompilevané verzi, proto je nutné si jej zkompilevat sám.

Předpoklady:

- Git (pouze ke stažení projektu z GitHubu)
- Funkční C++ compiler (clang++ nebo g++)
- Make
- LLVM
- LLD
- Příkazový řádek

Krok 1: Nejprve je důležité mít kód stažený. Zde použijeme rozhraní Gitu pro příkazový řádek, ale je také možné využít různých grafických rozhraní, která dokáží udělat to stejné. Alternativou ke Gitu by bylo si projekt z GitHubu stáhnout jako .zip archiv. Otevřeme si tedy náš příkazový řádek a ke stažení si projektu použijeme příkaz:

```
git clone https://github.com/F100cTomas/Lang.git
```

Stažený kód by se nyní měl nacházet ve složce Lang/. Pro další kroky budeme chtít přejít do složky s Makefile souborem:

```
cd Lang/cppcompiler/
```

Krok 2: Nyní ze složky cppcompiler/ spustíme make. Příkazu make je možné poskytovat argumenty. V aktuální verzi je možné poskytnout pouze dva. První je BUILD, kterým specifikujeme, že nechceme, aby výsledný soubor obsahoval debugovací symboly. Druhý je CXX, který říká, jaký C++ compiler bude použit. Protože výchozí hodnota pro BUILD je debug, doporučuji specifikovat hodnotu release. Pro rychlejší kompilaci použijeme argument -j8, který práci rozdělí mezi více vláken. Příkaz vypadá takhle:

```
make -j8 BUILD=release CXX=clang++
```

Krok 3: Vzniklý soubor se jmenuje `lang`. Pro spuštění compileru použijeme:

```
./lang code
```

Pro vypsaní tabulky argumentů `lang` existuje:

```
./lang code -h
```

Pokud chcete, aby byl příkaz `lang` přístupný všude, můžete poté soubor přesunout do složky `/usr/bin/`, nebo jiné složky v proměnné prostředí `PATH`, nebo složku s compilerem do `PATH` přidat.

3.1.2 Kompilace na Windows

Kompilace na Windows je téměř úplně stejná, jak na Linuxu. Stačí pouze nahradit Linux příkazový řádek příkazovým řádkem MinGW. Důležité je mít všechen předpokládaný software stažený uvnitř prostředí MinGW.

3.2 Uživatelské rozhraní

Uživatel s projektem interaguje pomocí příkazového řádku. Jako první argument uvádí vždy uživatel soubor, který bude zkompilován. Dále může uživatel specifikovat název vzniklého souboru pomocí možnosti `-o <název souboru>`. `-a` způsobí, že místo spustitelného souboru uživatel dostane instrukce procesoru ve formátu Assembly. `-d` vypíše do konzole mnoho nezajímavého textu, který slouží hlavně pro debugování compileru, nikoli uživatelských programů.

USAGE:

```
./lang <vstup> [OPTIONS]
```

ARGUMENTS:

<code><vstup></code>	Soubor ke kompilaci
----------------------------	---------------------

OPTIONS:

<code>-o <výstup></code>	Název zkompilovaného souboru (default: program)
<code>-a</code>	Assembly výstup
<code>-d</code>	Debug výstup
<code>-h</code>	Seznam argumentů

Obrázek 3.1: Nápověď, kterou uživatelské rozhraní poskytuje

3.3 Funkce

Každý program začíná ve funkci `main()`. Návrátová hodnota `main()` koresponduje s kódem ukončení programu. Tohle je nejjednodušší možný program, hned se ukončí s kódem 0.

```
fn main() : 0;
```

Pro tisknutí na obrazovku stačí zadat funkci putchar ASCII hodnotu požadovaného znaku. Následující program tedy vytiskne na obrazovku „A“:

```
fn main() {
    putchar 65;
    putchar 10;
    0
};
```

Každý znak má v této tabulce určitou hodnotu, putchar na základě toho ten určitý znak vypíše.

Číslo	Znak	Číslo	Znak	Číslo	Znak	Číslo	Znak
0	NUL	32	SPACE	64	@	96	'
1	SOH	33	!	65	A	97	a
2	STX	34	"	66	B	98	b
3	ETX	35	#	67	C	99	c
4	EOT	36	\$	68	D	100	d
5	ENQ	37	%	69	E	101	e
6	ACK	38	&	70	F	102	f
7	BEL	39	'	71	G	103	g
8	BS	40	(	72	H	104	h
9	TAB	41	)	73	I	105	i
10	LF	42	*	74	J	106	j
11	VT	43	+	75	K	107	k
12	FF	44	,	76	L	108	l
13	CR	45	-	77	M	109	m
14	SO	46	.	78	N	110	n
15	SI	47	/	79	O	111	o
16	DLE	48	0	80	P	112	p
17	DC1	49	1	81	Q	113	q
18	DC2	50	2	82	R	114	r
19	DC3	51	3	83	S	115	s
20	DC4	52	4	84	T	116	t
21	NAK	53	5	85	U	117	u
22	SYN	54	6	86	V	118	v
23	ETB	55	7	87	W	119	w
24	CAN	56	8	88	X	120	x
25	EM	57	9	89	Y	121	y
26	SUB	58	:	90	Z	122	z
27	ESC	59	;	91	[	123	{
28	FS	60	<	92	\	124	
29	GS	61	=	93	]	125	}
30	RS	62	>	94	^	126	~
31	US	63	?	95	_	127	DEL

Tabulka 3.1: ASCII

```
fn num(n) : {
    if (n >= 10) {
        num(n / 10);
    };
    putchar(48+n%10);
};
```

Funkce mohou mít parametr a být rekurzivní.

3.4 Operátory

Jazyk podporuje jednoduché aritmetické operátory, (+, −, *, /, %) porovnávací operátory, (==, !=, >=, >, <=, <) a přiřazovací operátor. (=)

3.5 Klíčová slova if a while

Klíčová slova if i while se chovají, jak by člověk očekával. Možná někoho zarazí, že if vrací hodnotu na základě toho, jestli byla podmínka pravdivá.

3.6 Proměnné

```
let global = 66;
fn main() {
    let local = 9;
    global = global - 1;
    local = local + 1;
    putchar global;
    putchar local;
};
```

Existují lokální i globální proměnné. Jejich hodnoty se mohou měnit.

3.7 Ukázkové programy

FizzBuzz

```
fn fizz(f) : {
  if (f % 3 == 0) {
    putchar (102);
    putchar (105);
    putchar (122);
    putchar (122);
  } else 0;
};
fn buzz(b) : {
  if (b % 5 == 0) {
    putchar (98);
    putchar (117);
    putchar (122);
    putchar (122);
  } else 0;
};
fn number(num) : {
  if (num >= 10) {
    number (num / 10);
  } else 0;
  putchar (48 + num % 10);
};
fn fizzbuzz1(n1) : {
  if (n1 % 3 != 0) {
    fizzbuzz2 (n1);
  } else 0;
};
fn fizzbuzz2(n2) : {
  if (n2 % 5 != 0) {
    number (n2);
  } else 0;
};
fn main() : {
  let i = 1;
  while (i < 100) {
    fizz (i);
    buzz (i);
    fizzbuzz1 (i);
    putchar (10);
    i = i + 1;
  };
};
```

Fibonacciho posloupnost

```
fn print1() : {
  putchar 32;
  putchar 49;
  putchar 10;
};

fn number(num) : {
  if (num >= 10) {
    number (num / 10);
  } else 0;
  putchar (48 + num % 10);
};

let a1 = 1;
let a2 = 1;

fn main() : {
  let i = 0;
  print1();
  print1();
  while (i < 50) {
    let a3 = a1 + a2;
    putchar 32;
    number a3;
    putchar 10;
    a1 = a2;
    a2 = a3;
    i = i + 1;
  };
};
```

4 Závěr

S výsledkem svého projektu jsem spokojen. Všechny hlavních cílů se mi podařilo včas dosáhnout. Naopak s vedlejšími cíli jsem byl příliš ambiciózní, což jsem i čekal, protože jsem si spíše dal při výběru vedlejších cílů prostor, abych mohl projekt rozvíjet různými směry. Upřímně jsem trochu doufal, že stihnu i typový systém, protože typové systémy jsou velmi důležitou součástí jazyků, ale bohužel tomu tak nebylo. Proto bude typový systém mou hlavní prioritou při budoucí práci na projektu.

Kromě toho je prostoru na rozvoj velké množství. Jedním obrovským tématem, kterého jsem se ani nedotkl, jsou optimalizace. Jiným milníkem, od kterého je jazyk stále velmi daleko, je OOP. Nejraději bych, aby se jazyk dostal tak daleko, že mohu jeho compiler přepsat ve stejném jazyce, ale to v momentální době ještě nevypadá realisticky. Tyto cíle však spadají do daleké budoucnosti a krátkodobě se budu zaměřovat spíše na zvládnutí toho, co jsem určil jako vedlejší cíle a ještě není hotové.

Zdroje

1. SUK, Ondřej. *GitHub - F100cTomas/Lang: Maturitní Projekt 2026*. 2025. Dostupné také z: <https://github.com/F100cTomas/Lang>.
2. LLVM PROJECT. *My First Language Frontend with LLVM Tutorial* [online]. 2026. [cit. 2026-05-02]. Dostupné z: <https://llvm.org/docs/tutorial/MyFirstLanguageFrontend/index.html>.